NAME: MANSHI VERMA

CUID: CU24250193

ROLL NO: 30

COMPUTER VISION LAB

LABSHEET 5: SIFT AND HOG FEATURE DESCRIPTORS

NAME: MANSHI VERMA
ROLL NO: 30

1.Build a keypoint detector and visualizer using SIFT ,displaying detcted keypoints with their scale and orientation.

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

print("■ Upload an image:")
uploaded = files.upload()

if not uploaded:
    raise ValueError("No image was uploaded.")

input_path = list(uploaded.keys())[0]

image = cv2.imread(input_path)

if image is None:
    raise ValueError("Unable to read the image.")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)

print("\n========== SIFT RESULTS ==========")
print("Image:", input_path)
print("Image Size:", image.shape[1], "x", image.shape[0])
print("Number of Keypoints Detected:", len(keypoints))

print("\nFirst 20 Keypoints:")
print("-" * 70)
print("No.\tX\tY\tScale(Size)\tOrientation")

for i, kp in enumerate(keypoints[:20]):
    x, y = kp.pt
    scale = kp.size
    orientation = kp.angle

    print(
        f"{i+1}\t"
        f"{x:.2f}\t"
        f"{y:.2f}\t"
        f"{scale:.2f}\t\t"
        f"{orientation:.2f}°"
    )

output_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(10, 7))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()

plt.figure(figsize=(12, 8))
plt.imshow(output_image)
plt.title(
    f"SIFT Keypoints\n"
    f"Total Keypoints: {len(keypoints)}\n"
    f"Circle Size = Scale | Line Direction = Orientation"
)
plt.axis("off")
plt.show()

output_path = "sift_keypoints_visualization.jpg"

output_bgr = cv2.cvtColor(output_image, cv2.COLOR_RGB2BGR)

cv2.imwrite(output_path, output_bgr)

print("\n■ SIFT visualization saved as:", output_path)

files.download(output_path)
```

■ Upload an image:

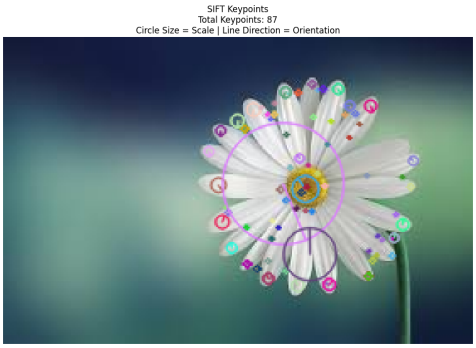
Saving images (5).jpg to images (5) (1).jpg

========== SIFT RESULTS ==========
Image: images (5) (1).jpg
Image Size: 278 x 181
Number of Keypoints Detected: 87

First 20 Keypoints:

No. X Y Scale(Size) Orientation
1 121.37 66.91 5.53 70.14°
2 127.00 86.87 8.70 55.34°
3 128.31 54.82 5.51 91.08°
4 129.27 108.55 8.05 290.18°
5 131.36 99.95 1.90 63.61°
6 132.61 102.97 3.43 64.32°
7 134.81 123.96 5.83 260.88°
8 138.26 48.98 7.76 15.59°
9 138.26 48.98 7.76 85.05°
10 142.63 53.77 4.45 111.04°
11 142.63 53.77 4.45 346.51°
12 143.62 111.83 2.42 49.58°
13 143.62 111.83 2.42 249.20°
14 144.06 133.40 4.61 13.03°
15 144.06 133.40 4.61 247.24°
16 147.83 42.37 8.01 101.34°
17 149.94 136.33 4.19 240.29°
18 149.94 136.33 4.19 357.06°
19 152.68 40.92 1.81 129.11°
20 156.15 45.18 2.12 139.31°





■ SIFT visualization saved as: sift_keypoints_visualization.jpg

NAME:MASNHI VERMA

ROLL NO:30

2. Develop an image matching tool that uses SIFT descriptors to find and draw correspondences between two photos of the same scene.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("/content/images (6).jpg")
uploaded1 = files.upload()

if not uploaded1:
    raise ValueError("First image was not uploaded.")

image1_path = list(uploaded1.keys())[0]

print("■ First image uploaded:", image1_path)

print("/content/download.jpg")
uploaded2 = files.upload()

if not uploaded2:
    raise ValueError("Second image was not uploaded.")

image2_path = list(uploaded2.keys())[0]

print("■ Second image uploaded:", image2_path)

image1 = cv2.imread(image1_path)
image2 = cv2.imread(image2_path)

if image1 is None:
    raise ValueError("Could not read first image.")

if image2 is None:
    raise ValueError("Could not read second image.")

gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()

keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("\n========== SIFT RESULTS ==========")
print("Image 1:", image1_path)
print("Image 2:", image2_path)
print("Keypoints in Image 1:", len(keypoints1))
print("Keypoints in Image 2:", len(keypoints2))

bf = cv2.BFMatcher(cv2.NORM_L2)

matches = bf.knnMatch(
    descriptors1,
    descriptors2,
    k=2
)

good_matches = []

ratio_threshold = 0.75

for m, n in matches:
    if m.distance < ratio_threshold * n.distance:
        good_matches.append(m)

print("\n========== MATCHING RESULTS ==========")
print("Total matches:", len(matches))
print("Good matches:", len(good_matches))

match_image = cv2.drawMatches(
    image1,
    keypoints1,
    image2,
    keypoints2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 7))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis("off")

plt.show()

plt.figure(figsize=(18, 9))

plt.imshow(
    cv2.cvtColor(match_image, cv2.COLOR_BGR2RGB)
)

plt.title(
    f"SIFT Image Matching - {len(good_matches)} Good Matches"
)

plt.axis("off")
plt.show()

output_path = "sift_image_matching.jpg"

cv2.imwrite(output_path, match_image)

print("\n■ Output saved:", output_path)

files.download(output_path)
```

/content/images (6).jpg

Saving images (6).jpg to images (6) (1).jpg
■ First image uploaded: images (6) (1).jpg

/content/download.jpg

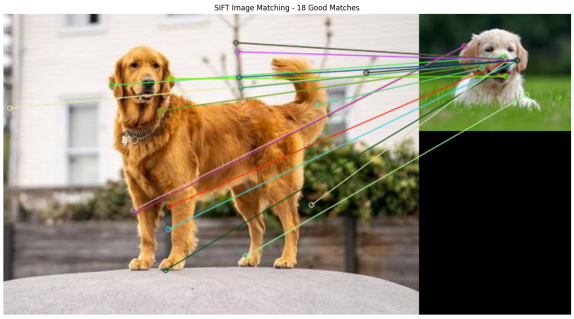
Saving download.jpg to download (1).jpg
■ Second image uploaded: download (1).jpg

========== SIFT RESULTS ==========
Image 1: images (6) (1).jpg
Image 2: download (1).jpg
Keypoints in Image 1: 962
Keypoints in Image 2: 153

========== MATCHING RESULTS ==========
Total matches: 962
Good matches: 18







■ Output saved: sift_image_matching.jpg

NAME:MASNHI VERMA

ROLL NO:30

3. Create a simple panorama stitcher using SIFT feature matching and homography estimation.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("/content/images (4).jpg")
uploaded1 = files.upload()

if not uploaded1:
    raise ValueError("First image was not uploaded.")

image1_path = list(uploaded1.keys())[0]

print("■ First image:", image1_path)

print("/content/download.jpg")
uploaded2 = files.upload()

if not uploaded2:
    raise ValueError("Second image was not uploaded.")

image2_path = list(uploaded2.keys())[0]

print("■ Second image:", image2_path)

image1 = cv2.imread(image1_path)
image2 = cv2.imread(image2_path)

if image1 is None or image2 is None:
    raise ValueError("Could not read one or both images.")

sift = cv2.SIFT_create()

gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

keypoints1, descriptors1 = sift.detectAndCompute(
    gray1, None
)

keypoints2, descriptors2 = sift.detectAndCompute(
    gray2, None
)

print("\n========== SIFT ==========")
print("Keypoints in Image 1:", len(keypoints1))
print("Keypoints in Image 2:", len(keypoints2))

bf = cv2.BFMatcher(cv2.NORM_L2)

matches = bf.knnMatch(
    descriptors1,
    descriptors2,
    k=2
)

good_matches = []

for m, n in matches:

    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good matches:", len(good_matches))

if len(good_matches) < 4:
    raise ValueError(
        "Not enough good matches to calculate homography."
    )

src_points = np.float32([
    keypoints1[m.queryIdx].pt
    for m in good_matches
]).reshape(-1, 1, 2)

dst_points = np.float32([
    keypoints2[m.trainIdx].pt
    for m in good_matches
]).reshape(-1, 1, 2)

H, mask = cv2.findHomography(
    src_points,
    dst_points,
    cv2.RANSAC,
    5.0
)

if H is None:
    raise ValueError("Homography could not be estimated.")

inliers = mask.ravel().sum()

print("\n========== HOMOGRAPHY ==========")
print("Homography Matrix:")
print(H)
print("Inlier Matches:", inliers)

h1, w1 = image1.shape[:2]
h2, w2 = image2.shape[:2]

corners1 = np.float32([
    [0, 0],
    [w1, 0],
    [w1, h1],
    [0, h1]
]).reshape(-1, 1, 2)

corners2 = np.float32([
    [0, 0],
    [w2, 0],
    [w2, h2],
    [0, h2]
]).reshape(-1, 1, 2)

transformed_corners1 = cv2.perspectiveTransform(
    corners1,
    H
)

all_corners = np.concatenate(
    (transformed_corners1, corners2),
    axis=0
)

[x_min, y_min] = np.int32(
    all_corners.min(axis=0).ravel() - 0.5
)

[x_max, y_max] = np.int32(
    all_corners.max(axis=0).ravel() + 0.5
)

translation = np.array([
    [1, 0, -x_min],
    [0, 1, -y_min],
    [0, 0, 1]
])

panorama_width = x_max - x_min
panorama_height = y_max - y_min

panorama = cv2.warpPerspective(
    image1,
    translation @ H,
    (panorama_width, panorama_height)
)

panorama[
    -y_min:-y_min + h2,
    -x_min:-x_min + w2
] = image2

gray_panorama = cv2.cvtColor(
    panorama,
    cv2.COLOR_BGR2GRAY
)

_, threshold = cv2.threshold(
    gray_panorama,
    1,
    255,
    cv2.THRESH_BINARY
)

contours, _ = cv2.findContours(
    threshold,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

if contours:

    largest_contour = max(
        contours,
        key=cv2.contourArea
    )

    x, y, w, h = cv2.boundingRect(
        largest_contour
    )

    panorama = panorama[
        y:y+h,
        x:x+w
    ]

plt.figure(figsize=(16, 6))

plt.subplot(1, 2, 1)
plt.imshow(
    cv2.cvtColor(image1, cv2.COLOR_BGR2RGB)
)
plt.title("Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(
    cv2.cvtColor(image2, cv2.COLOR_BGR2RGB)
)
plt.title("Image 2")
plt.axis("off")

plt.show()

match_visualization = cv2.drawMatches(
    image1,
    keypoints1,
    image2,
    keypoints2,
    good_matches,
    None,
    matchesMask=mask.ravel().tolist(),
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(18, 8))

plt.imshow(
    cv2.cvtColor(
        match_visualization,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    f"SIFT Matches - {inliers} Inlier Matches"
)

plt.axis("off")
plt.show()

plt.figure(figsize=(18, 8))

plt.imshow(
    cv2.cvtColor(
        panorama,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Final Stitched Panorama")
plt.axis("off")
plt.show()

output_path = "stitched_panorama.jpg"

cv2.imwrite(
    output_path,
    panorama
)

print("\n■ Panorama saved as:", output_path)

files.download(output_path)
```

/content/images (4).jpg

Saving images (4).jpg to images (4) (4).jpg
■ First image: images (4) (4).jpg
/content/download.jpg

Saving download.jpg to download (2).jpg
■ Second image: download (2).jpg

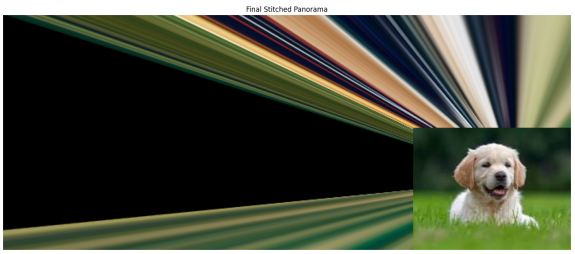
========== SIFT ==========
Keypoints in Image 1: 745
Keypoints in Image 2: 153
Good matches: 17

========== HOMOGRAPHY ==========
Homography Matrix:
[[-2.43491249e-01 -3.69423124e-01 1.28630690e+02]
[-1.13806030e-01 -1.80331255e-01 6.17537870e+01]
[-1.85152496e-03 -3.01424975e-03 1.00000000e+00]]
Inlier Matches: 8









■ Panorama saved as: stitched_panorama.jpg

NAME:MASNHI VERMA

ROLL NO:30

4. Build a logo or object locator that finds a reference object inside a cluttered scene using SIFT feature matching.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("/content/images (7).jpg")
print("Example: clear photo of the logo/object")

uploaded_ref = files.upload()

if not uploaded_ref:
    raise ValueError("Reference image was not uploaded.")

reference_path = list(uploaded_ref.keys())[0]

print("■ Reference image:", reference_path)

print("/content/images (4).jpg")
print("Example: image containing the reference logo/object")

uploaded_scene = files.upload()

if not uploaded_scene:
    raise ValueError("Scene image was not uploaded.")

scene_path = list(uploaded_scene.keys())[0]

print("■ Scene image:", scene_path)

reference = cv2.imread(reference_path)
scene = cv2.imread(scene_path)

if reference is None:
    raise ValueError("Could not read reference image.")

if scene is None:
    raise ValueError("Could not read scene image.")

gray_reference = cv2.cvtColor(
    reference,
    cv2.COLOR_BGR2GRAY
)

gray_scene = cv2.cvtColor(
    scene,
    cv2.COLOR_BGR2GRAY
)

sift = cv2.SIFT_create()

keypoints_ref, descriptors_ref = sift.detectAndCompute(
    gray_reference,
    None
)

keypoints_scene, descriptors_scene = sift.detectAndCompute(
    gray_scene,
    None
)

print("\n========== SIFT RESULTS ==========")
print(
    "Reference keypoints:",
    len(keypoints_ref)
)

print(
    "Scene keypoints:",
    len(keypoints_scene)
)

bf = cv2.BFMatcher(cv2.NORM_L2)

matches = bf.knnMatch(
    descriptors_ref,
    descriptors_scene,
    k=2
)

good_matches = []

ratio_threshold = 0.75

for m, n in matches:

    if m.distance < ratio_threshold * n.distance:
        good_matches.append(m)

print("\n========== MATCHING ==========")
print("Total matches:", len(matches))
print("Good matches:", len(good_matches))

if len(good_matches) < 4:

    raise ValueError(
        "Not enough good matches. "
        "Try clearer images with more visible features."
    )

src_points = np.float32([
    keypoints_ref[m.queryIdx].pt
    for m in good_matches
]).reshape(-1, 1, 2)

dst_points = np.float32([
    keypoints_scene[m.trainIdx].pt
    for m in good_matches
]).reshape(-1, 1, 2)

H, mask = cv2.findHomography(
    src_points,
    dst_points,
    cv2.RANSAC,
    5.0
)

if H is None:
    raise ValueError(
        "Could not estimate homography."
    )

inliers = mask.ravel().sum()

print("\n========== HOMOGRAPHY ==========")
print("Inlier matches:", inliers)

print("\nHomography Matrix:")
print(H)

h, w = gray_reference.shape

reference_corners = np.float32([
    [0, 0],
    [w, 0],
    [w, h],
    [0, h]
]).reshape(-1, 1, 2)

scene_corners = cv2.perspectiveTransform(
    reference_corners,
    H
)

result = scene.copy()

cv2.polylines(
    result,
    [np.int32(scene_corners)],
    True,
    (0, 255, 0),
    4,
    cv2.LINE_AA
)

for match, is_inlier in zip(
    good_matches,
    mask.ravel()
):

    if is_inlier:

        point = keypoints_scene[
            match.trainIdx
        ].pt

        x = int(point[0])
        y = int(point[1])

        cv2.circle(
            result,
            (x, y),
            5,
            (0, 0, 255),
            -1
        )

cv2.putText(
    result,
    f"Object Detected | Inliers: {inliers}",
    (20, 40),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (0, 255, 0),
    2
)

plt.figure(figsize=(7, 6))

plt.imshow(
    cv2.cvtColor(
        reference,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Reference Object / Logo")
plt.axis("off")
plt.show()

match_image = cv2.drawMatches(
    reference,
    keypoints_ref,
    scene,
    keypoints_scene,
    good_matches,
    None,
    matchesMask=mask.ravel().tolist(),
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(18, 9))

plt.imshow(
    cv2.cvtColor(
        match_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    f"SIFT Matches | Inliers: {inliers}"
)

plt.axis("off")
plt.show()

plt.figure(figsize=(12, 8))

plt.imshow(
    cv2.cvtColor(
        result,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    "Detected Object / Logo in Cluttered Scene"
)

plt.axis("off")
plt.show()

output_path = "object_locator_result.jpg"

cv2.imwrite(
    output_path,
    result
)

print("\n■ Detection result saved as:")
print(output_path)

files.download(output_path)
```

/content/images (7).jpg
Example: clear photo of the logo/object

Saving images (7).jpg to images (7) (2).jpg
■ Reference image: images (7) (2).jpg
/content/images (4).jpg
Example: image containing the reference logo/object

Saving images (4).jpg to images (4) (5).jpg
■ Scene image: images (4) (5).jpg

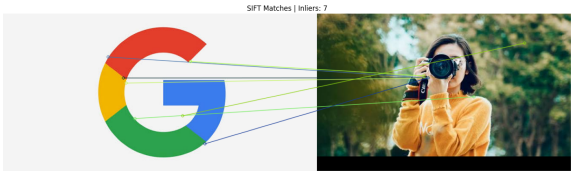
========== SIFT RESULTS ==========
Reference keypoints: 123
Scene keypoints: 745

========== MATCHING ==========
Total matches: 123
Good matches: 15

========== HOMOGRAPHY ==========
Inlier matches: 7

Homography Matrix:
[[-1.22092971e-01 -8.79529347e-01 2.61892300e+02]
[-6.77272253e-02 -5.12419107e-01 1.51109585e+02]
[-3.97277164e-04 -3.48047132e-03 1.00000000e+00]]





Detected Object / Logo in Cluttered Scene



■ Detection result saved as:
object_locator_result.jpg

NAME:MASNHI VERMA

ROLL NO:30

5. Develop a HOG based pedestrain or person detector demo ona set of sample images.

```
!pip install -q scikit-image scikit-learn

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
import os
import zipfile

from skimage.feature import hog
from skimage.transform import resize
from sklearn.svm import SVC

print("■ Upload sample image:")

uploaded = files.upload()

if not uploaded:
    raise ValueError("No image was uploaded.")

filename = list(uploaded.keys())[0]

print("\n■ Image uploaded:", filename)

image = cv2.imread(filename)

if image is None:
    raise ValueError("Could not read the image.")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

print("Image size:", image_rgb.shape)

gray = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2GRAY)

resized = resize(
    gray,
    (128, 64)
)

hog_features = hog(
    resized,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys'
)

print("\n■ HOG feature extraction completed.")

print("Number of HOG features:",
      len(hog_features))

features, hog_image = hog(
    resized,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys',
    visualize=True
)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image, cmap="gray")
plt.title("HOG Feature Visualization")
plt.axis("off")

plt.tight_layout()
plt.show()

output_dir = "hog_output"
os.makedirs(output_dir, exist_ok=True)

hog_output = os.path.join(
    output_dir,
    "hog_visualization.jpg"
)

plt.figure(figsize=(8, 6))
plt.imshow(hog_image, cmap="gray")
plt.title("HOG Feature Visualization")
plt.axis("off")
plt.savefig(hog_output, bbox_inches="tight")
plt.close()

print("\n■ HOG visualization saved:")
print(hog_output)

zip_filename = "hog_pedestrian_results.zip"

with zipfile.ZipFile(
    zip_filename,
    "w",
    zipfile.ZIP_DEFLATED
) as zipf:

    for file in os.listdir(output_dir):

        file_path = os.path.join(
            output_dir,
            file
        )

        zipf.write(
            file_path,
            arcname=file
        )

print("\n■ ZIP file created:")
print(zip_filename)

files.download(zip_filename)
```

■ Upload sample image:

Saving download.jpg to download (4).jpg

■ Image uploaded: download (4).jpg
Image size: (148, 192, 3)

■ HOG feature extraction completed.

Number of HOG features: 3780

Original Image



HOG Feature Visualization



■ HOG visualization saved:
hog_output/hog_visualization.jpg

■ ZIP file created:
hog_pedestrian_results.zip

NAME:MASNHI VERMA

ROLL NO:30

6. Create a HOG visualizer that overlays the gradient orientation histogram grid on top of an image.

```
!pip install -q scikit-image

import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files
from skimage.feature import hog
from skimage.transform import resize

print("■ Upload an image:")

uploaded = files.upload()

if not uploaded:
    raise ValueError("No image was uploaded.")

filename = list(uploaded.keys())[0]

print("\n■ Image uploaded:", filename)

image = cv2.imread(filename)

if image is None:
    raise ValueError("Could not read the image.")

image_rgb = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)

gray = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2GRAY
)

height, width = gray.shape

scale = min(600 / width, 600 / height, 1.0)

new_width = int(width * scale)
new_height = int(height * scale)

gray_small = cv2.resize(
    gray,
    (new_width, new_height)
)

rgb_small = cv2.resize(
    image_rgb,
    (new_width, new_height)
)

gx = cv2.Sobel(
    gray_small,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)

gy = cv2.Sobel(
    gray_small,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

magnitude = np.sqrt(
    gx ** 2 + gy ** 2
)

orientation = np.degrees(
    np.arctan2(gy, gx)
)

orientation = orientation % 180

cell_size = 16

rows = new_height // cell_size
cols = new_width // cell_size

print("\nImage size:", new_width, "x", new_height)
print("HOG cell size:", cell_size)
print("Grid:", rows, "rows x", cols, "columns")

overlay = rgb_small.copy()

overlay = overlay.astype(np.float32)

for r in range(rows + 1):

    y = r * cell_size

    if y < new_height:

        cv2.line(
            overlay,
            (0, y),
            (new_width, y),
            (255, 255, 0),
            1
        )

for c in range(cols + 1):

    x = c * cell_size

    if x < new_width:

        cv2.line(
            overlay,
            (x, 0),
            (x, new_height),
            (255, 255, 0),
            1
        )

num_bins = 9

bin_width = 180 / num_bins

for r in range(rows):

    for c in range(cols):

        y1 = r * cell_size
        y2 = y1 + cell_size

        x1 = c * cell_size
        x2 = x1 + cell_size

        cell_mag = magnitude[
            y1:y2,
            x1:x2
        ]

        cell_ori = orientation[
            y1:y2,
            x1:x2
        ]

        hist = np.zeros(num_bins)
        for i in range(cell_mag.shape[0]):
```

```
            for j in range(cell_mag.shape[1]):

                angle = cell_ori[i, j]
                mag = cell_mag[i, j]

                bin_index = int(
                    angle // bin_width
                )

                if bin_index >= num_bins:
                    bin_index = num_bins - 1

                hist[bin_index] += mag

        if hist.max() > 0:
            hist = hist / hist.max()

        center_x = x1 + cell_size // 2
        center_y = y1 + cell_size // 2

        max_length = cell_size * 0.45

        for b in range(num_bins):

            angle = (
                b * bin_width
                + bin_width / 2
            )

            angle_rad = np.radians(angle)

            length = (
                hist[b] * max_length
            )

            dx = np.cos(angle_rad) * length
            dy = np.sin(angle_rad) * length

            pt1 = (
                int(center_x - dx),
                int(center_y - dy)
            )

            pt2 = (
                int(center_x + dx),
                int(center_y + dy)
            )

            cv2.line(
                overlay,
                pt1,
                pt2,
                (255, 0, 0),
                1
            )

overlay = np.clip(
    overlay,
    0,
    255
).astype(np.uint8)

result = cv2.addWeighted(
    rgb_small,
    0.55,
    overlay,
    0.45,
    0
)

plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)

plt.imshow(rgb_small)

plt.title("Original Image")

plt.axis("off")

plt.subplot(1, 2, 2)

plt.imshow(result)

plt.title(
    "HOG Gradient Orientation Grid Overlay"
)

plt.axis("off")

plt.tight_layout()

plt.show()

output_filename = "hog_orientation_grid_overlay.jpg"

result_bgr = cv2.cvtColor(
    result,
    cv2.COLOR_RGB2BGR
)

cv2.imwrite(
    output_filename,
    result_bgr
)

print("\n■ HOG visualization saved:")
print(output_filename)

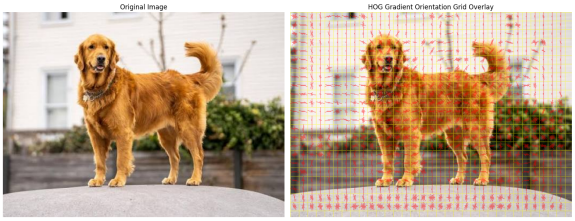
files.download(output_filename)
```

■ Upload an image:

Saving images (6).jpg to images (6) (4).jpg

■ Image uploaded: images (6) (4).jpg

Image size: 525 x 380
HOG cell size: 16
Grid: 23 rows x 32 columns



■ HOG visualization saved:
hog_orientation_grid_overlay.jpg

NAME:MASNHI VERMA

ROLL NO:30

7. Buid a duplicate / near duplicate using SIFT feature similarity scoring.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("■ Upload FIRST image:")

uploaded1 = files.upload()

if not uploaded1:
    raise ValueError("First image was not uploaded.")

file1 = list(uploaded1.keys())[0]

print("■ First image:", file1)

print("\n■ Upload SECOND image:")

uploaded2 = files.upload()

if not uploaded2:
    raise ValueError("Second image was not uploaded.")

file2 = list(uploaded2.keys())[0]

print("■ Second image:", file2)

image1 = cv2.imread(file1)
image2 = cv2.imread(file2)

if image1 is None:
    raise ValueError("Could not read first image.")

if image2 is None:
    raise ValueError("Could not read second image.")

gray1 = cv2.cvtColor(
    image1,
    cv2.COLOR_BGR2GRAY
)

gray2 = cv2.cvtColor(
    image2,
    cv2.COLOR_BGR2GRAY
)

sift = cv2.SIFT_create()

keypoints1, descriptors1 = (
    sift.detectAndCompute(
        gray1,
        None
    )
)

keypoints2, descriptors2 = (
    sift.detectAndCompute(
        gray2,
        None
    )
)

print("\n======================================")
print("SIFT FEATURE INFORMATION")
print("======================================")

print(
    "Image 1 keypoints:",
    len(keypoints1)
)

print(
    "Image 2 keypoints:",
    len(keypoints2)
)

if descriptors1 is None:
    raise ValueError(
        "No SIFT features found in first image."
    )

if descriptors2 is None:
    raise ValueError(
        "No SIFT features found in second image."
    )

bf = cv2.BFMatcher(
    cv2.NORM_L2
)

matches = bf.knnMatch(
    descriptors1,
    descriptors2,
    k=2
)

good_matches = []

for pair in matches:

    if len(pair) < 2:
        continue

    m, n = pair

    if m.distance < 0.75 * n.distance:

        good_matches.append(m)

print(
    "\nGood matches:",
    len(good_matches)
)

min_keypoints = min(
    len(keypoints1),
    len(keypoints2)
)

if min_keypoints == 0:

    similarity_score = 0

else:

    similarity_score = (
        len(good_matches) /
        min_keypoints
    ) * 100

if len(good_matches) > 0:

    average_distance = np.mean(
        [
            m.distance
            for m in good_matches
        ]
    )

else:

    average_distance = 999

if similarity_score >= 50:

    result = "DUPLICATE"

elif similarity_score >= 20:

    result = "NEAR-DUPLICATE"

else:

    result = "DIFFERENT"

print("\n")
print("================================================")
print("          SIFT DUPLICATE DETECTION RESULT")
print("================================================")

print(
    f"Image 1              : {file1}"
)

print(
    f"Image 2              : {file2}"
)

print(
    f"SIFT Keypoints 1     : {len(keypoints1)}"
)

print(
    f"SIFT Keypoints 2     : {len(keypoints2)}"
)

print(
    f"Good Matches          : {len(good_matches)}"
)

print(
    f"Similarity Score      : "
    f"{similarity_score:.2f}%"
)

print(
    f"Average Match Distance: "
    f"{average_distance:.2f}"
)

print(
    f"Final Result          : {result}"
)

print("================================================")

matches_to_draw = good_matches[:50]

match_image = cv2.drawMatches(
    image1,
    keypoints1,
    image2,
    keypoints2,
    matches_to_draw,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 8))

plt.imshow(
    cv2.cvtColor(
        match_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    f"SIFT Matches | "
    f"Similarity: {similarity_score:.2f}% | "
    f"Result: {result}"
)

plt.axis("off")

plt.show()

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)

plt.imshow(
    cv2.cvtColor(
        image1,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Image 1")
```

```
plt.axis("off")

plt.subplot(1, 2, 2)

plt.imshow(
    cv2.cvtColor(
        image2,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Image 2")

plt.axis("off")

plt.tight_layout()

plt.show()

print("\n■ Duplicate / Near-Duplicate detection completed.")

if result == "DUPLICATE":

    print(
        "■ The two images are highly similar "
        "and are classified as DUPLICATE."
    )

elif result == "NEAR-DUPLICATE":

    print(
        "■ The two images are similar "
        "but have some differences."
    )

else:

    print(
        "■ The two images are sufficiently different."
    )
```

■ Upload FIRST image:

Saving download (1).jpg to download (1) (2).jpg
■ First image: download (1) (2).jpg

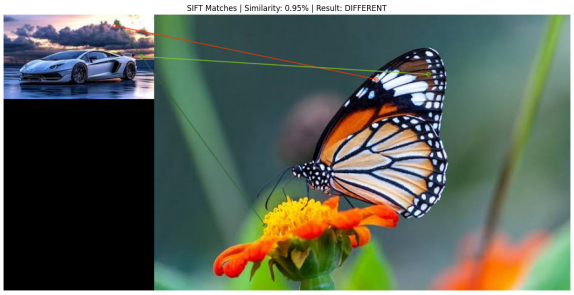
■ Upload SECOND image:

Saving images (2).jpg to images (2).jpg
■ Second image: images (2).jpg

```
====================================
SIFT FEATURE INFORMATION
====================================
Image 1 keypoints: 315
Image 2 keypoints: 1065

Good matches: 3


================================================
SIFT DUPLICATE DETECTION RESULT
================================================
Image 1 : download (1) (2).jpg
Image 2 : images (2).jpg
SIFT Keypoints 1 : 315
SIFT Keypoints 2 : 1065
Good Matches : 3
Similarity Score : 0.95%
Average Match Distance: 231.85
Final Result : DIFFERENT
================================================
```







■ Duplicate / Near-Duplicate detection completed.
■ The two images are sufficiently different.

NAME:MASNHI VERMA

ROLL NO:30

8. Devlop a rotation/scale invariance tester that verifies SIFT keypoints remain consisitent under image transformations.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("■ Upload ORIGINAL image:")

uploaded = files.upload()

if not uploaded:
    raise ValueError("No image was uploaded.")

filename = list(uploaded.keys())[0]

print("■ Image uploaded:", filename)

original = cv2.imread(filename)

if original is None:
    raise ValueError("Could not read the image.")

gray_original = cv2.cvtColor(
    original,
    cv2.COLOR_BGR2GRAY
)

height, width = gray_original.shape

center = (
    width // 2,
    height // 2
)

rotation_angle = 45

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    rotation_angle,
    1.0
)

rotated = cv2.warpAffine(
    original,
    rotation_matrix,
    (width, height)
)

gray_rotated = cv2.cvtColor(
    rotated,
    cv2.COLOR_BGR2GRAY
)

scale_factor = 1.5

scaled = cv2.resize(
    original,
    None,
    fx=scale_factor,
    fy=scale_factor,
    interpolation=cv2.INTER_LINEAR
)

gray_scaled = cv2.cvtColor(
    scaled,
    cv2.COLOR_BGR2GRAY
)

sift = cv2.SIFT_create()

kp_original, des_original = (
    sift.detectAndCompute(
        gray_original,
        None
    )
)

kp_rotated, des_rotated = (
    sift.detectAndCompute(
        gray_rotated,
        None
    )
)

kp_scaled, des_scaled = (
    sift.detectAndCompute(
        gray_scaled,
        None
    )
)

print("\n================================================")
print("SIFT KEYPOINT INFORMATION")
print("================================================")

print(
    "Original image keypoints :",
    len(kp_original)
)

print(
    "Rotated image keypoints  :",
    len(kp_rotated)
)

print(
    "Scaled image keypoints   :",
    len(kp_scaled)
)

if des_original is None:
    raise ValueError(
        "No SIFT descriptors found in original image."
    )

if des_rotated is None:
    raise ValueError(
        "No SIFT descriptors found in rotated image."
    )

if des_scaled is None:
    raise ValueError(
        "No SIFT descriptors found in scaled image."
    )

bf = cv2.BFMatcher(
    cv2.NORM_L2
)

matches_rotation = bf.knnMatch(
    des_original,
    des_rotated,
    k=2
)

good_rotation = []

for pair in matches_rotation:

    if len(pair) < 2:
        continue

    m, n = pair

    if m.distance < 0.75 * n.distance:

        good_rotation.append(m)

matches_scale = bf.knnMatch(
    des_original,
    des_scaled,
    k=2
)

good_scale = []

for pair in matches_scale:

    if len(pair) < 2:
        continue

    m, n = pair

    if m.distance < 0.75 * n.distance:

        good_scale.append(m)

rotation_score = (
    len(good_rotation) /
    min(
        len(kp_original),
        len(kp_rotated)
    )
) * 100

scale_score = (
    len(good_scale) /
    min(
        len(kp_original),
        len(kp_scaled)
    )
) * 100

print("\n================================================")
print("SIFT INVARIANCE TEST RESULTS")
print("================================================")

print(
    f"Rotation angle       : {rotation_angle} degrees"
)

print(
    f"Scale factor         : {scale_factor}x"
)

print(
    f"Rotation good matches: {len(good_rotation)}"
)

print(
    f"Scale good matches   : {len(good_scale)}"
)

print(
    f"Rotation score       : {rotation_score:.2f}%"
)

print(
    f"Scale score          : {scale_score:.2f}%"
)

print("================================================")

rotation_matches_image = cv2.drawMatches(
    original,
    kp_original,
    rotated,
    kp_rotated,
    good_rotation[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

scale_matches_image = cv2.drawMatches(
    original,
    kp_original,
    scaled,
    kp_scaled,
    good_scale[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 7))

plt.imshow(
    cv2.cvtColor(
        rotation_matches_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    f"SIFT Rotation Test | "
    f"{rotation_score:.2f}% similarity"
)

plt.axis("off")
plt.show()
```

```
plt.figure(figsize=(16, 7))

plt.imshow(
    cv2.cvtColor(
        scale_matches_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    f"SIFT Scale Test | "
    f"{scale_score:.2f}% similarity"
)

plt.axis("off")

plt.show()

original_keypoints = cv2.drawKeypoints(
    original,
    kp_original,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

rotated_keypoints = cv2.drawKeypoints(
    rotated,
    kp_rotated,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

scaled_keypoints = cv2.drawKeypoints(
    scaled,
    kp_scaled,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(
    cv2.cvtColor(
        original_keypoints,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Original SIFT Keypoints")
plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    cv2.cvtColor(
        rotated_keypoints,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Rotated SIFT Keypoints")
plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    cv2.cvtColor(
        scaled_keypoints,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Scaled SIFT Keypoints")
plt.axis("off")

plt.tight_layout()

plt.show()

print("\n")
print("================================================")
print("               FINAL CONCLUSION")
print("================================================")

if rotation_score >= 20:

    print(
        "■ SIFT shows good rotation invariance."
    )

else:

    print(
        "■■ Low rotation matching detected."
    )

if scale_score >= 20:

    print(
        "■ SIFT shows good scale invariance."
    )

else:

    print(
        "■■ Low scale matching detected."
    )

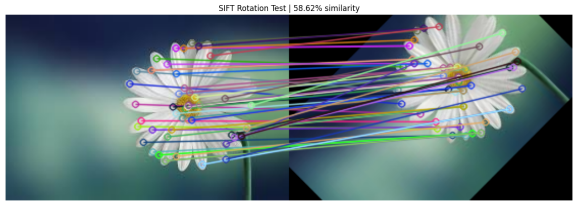
print("================================================")
```

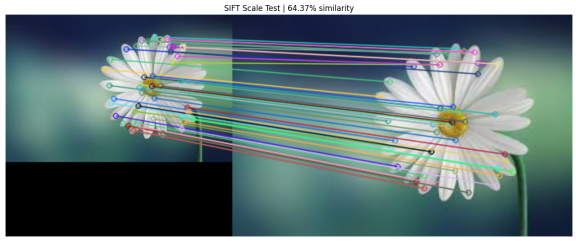
■ Upload ORIGINAL image:

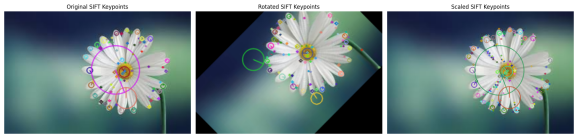
Saving images (5).jpg to images (5) (3).jpg
■ Image uploaded: images (5) (3).jpg

```
================================================
SIFT KEYPOINT INFORMATION
================================================
Original image keypoints : 87
Rotated image keypoints : 92
Scaled image keypoints : 164

================================================
SIFT INVARIANCE TEST RESULTS
================================================
Rotation angle : 45 degrees
Scale factor : 1.5x
Rotation good matches: 51
Scale good matches : 56
Rotation score : 58.62%
Scale score : 64.37%
================================================
```








```
================================================
FINAL CONCLUSION
================================================
■ SIFT shows good rotation invariance.
■ SIFT shows good scale invariance.
================================================
```

NAME:MASNHI VERMA

ROLL NO:30

9. Create a simple image-retrieval system that ranks a samll image database by SIFT feature similarity to a query image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

print("■ Upload QUERY IMAGE:")

query_uploaded = files.upload()

if not query_uploaded:
    raise ValueError("Query image was not uploaded.")

query_filename = list(
    query_uploaded.keys()
)[0]

print(
    "■ Query image:",
    query_filename
)

database_files = []

print("\n■ Now upload DATABASE IMAGES.")
print("You will upload 5 images one-by-one.")

for i in range(5):

    print(
        f"\n■ Upload database image {i+1}:"
    )

    uploaded = files.upload()

    if not uploaded:
        raise ValueError(
            f"Database image {i+1} was not uploaded."
        )

    filename = list(
        uploaded.keys()
    )[0]

    database_files.append(filename)

    print(
        f"■ Added: {filename}"
    )

query_image = cv2.imread(
    query_filename
)

if query_image is None:
    raise ValueError(
        "Could not read query image."
    )

sift = cv2.SIFT_create()

query_gray = cv2.cvtColor(
    query_image,
    cv2.COLOR_BGR2GRAY
)

query_keypoints, query_descriptors = (
    sift.detectAndCompute(
        query_gray,
        None
    )
)

if query_descriptors is None:
    raise ValueError(
        "No SIFT features found in query image."
    )

print("\n========================================")
print("QUERY IMAGE INFORMATION")
print("========================================")

print(
    "Query image:",
    query_filename
)

print(
    "Query keypoints:",
    len(query_keypoints)
)

bf = cv2.BFMatcher(
    cv2.NORM_L2
)

results = []

print("\n========================================")
print("SEARCHING DATABASE")
print("========================================")

for filename in database_files:

    image = cv2.imread(
        filename
    )

    if image is None:

        print(
            f"■■ Could not read {filename}"
        )

        continue

    gray = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2GRAY
    )

    keypoints, descriptors = (
        sift.detectAndCompute(
            gray,
            None
        )
    )

    if descriptors is None:

        print(
            f"■■ No SIFT features in {filename}"
        )

        continue

    matches = bf.knnMatch(
        query_descriptors,
        descriptors,
        k=2
    )

    good_matches = []

    for pair in matches:

        if len(pair) < 2:
            continue

        m, n = pair

        if m.distance < 0.75 * n.distance:

            good_matches.append(m)

    min_features = min(
        len(query_keypoints),
        len(keypoints)
    )

    if min_features > 0:

        similarity = (
            len(good_matches)
            /
            min_features
        ) * 100

    else:

        similarity = 0

    results.append({
        "filename": filename,
        "score": similarity,
        "matches": len(good_matches),
        "keypoints": len(keypoints)
```

```
    })

    print(
        f"{filename}: "
        f"{similarity:.2f}% similarity | "
        f"{len(good_matches)} good matches"
    )

results.sort(
    key=lambda x: x["score"],
    reverse=True
)

print("\n")
print("=============================================")
print("           SIFT IMAGE RETRIEVAL RESULTS")
print("=============================================")

for rank, result in enumerate(
    results,
    start=1
):

    print(
        f"Rank {rank}: "
        f"{result['filename']} | "
        f"Similarity: "
        f"{result['score']:.2f}% | "
        f"Good Matches: "
        f"{result['matches']}"
    )

print("=============================================")

plt.figure(
    figsize=(6, 5)
)

plt.imshow(
    cv2.cvtColor(
        query_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    "QUERY IMAGE"
)

plt.axis("off")

plt.show()

num_results = len(results)

plt.figure(
    figsize=(16, 10)
)

for i, result in enumerate(
    results
):

    image = cv2.imread(
        result["filename"]
    )

    image_rgb = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )

    plt.subplot(
        2,
        3,
        i + 1
    )

    plt.imshow(
        image_rgb
    )

    plt.title(
        f"Rank {i+1}\n"
        f"{result['score']:.2f}% similarity"
    )

    plt.axis("off")

plt.tight_layout()

plt.show()

names = [
    r["filename"]
    for r in results
]

scores = [
    r["score"]
    for r in results
]

plt.figure(
    figsize=(12, 6)
)

plt.bar(
    names,
    scores
)

plt.xlabel(
    "Database Images"
)

plt.ylabel(
    "SIFT Similarity Score (%)"
)

plt.title(
    "Image Retrieval Ranking"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()

if len(results) > 0:

    best = results[0]

    print("\n")
    print("=======================================")
    print("              BEST MATCH")
    print("=======================================")

    print(
        "Image:",
        best["filename"]
    )

    print(
        f"Similarity: "
        f"{best['score']:.2f}%"
    )

    print(
        "Good SIFT matches:",
        best["matches"]
    )

    print("=======================================")

print("\n■ IMAGE RETRIEVAL COMPLETED!")
```

■ Upload QUERY IMAGE:

Saving images.png to images.png
■ Query image: images.png

■ Now upload DATABASE IMAGES.
You will upload 5 images one-by-one.

■ Upload database image 1:

Saving images (8).jpg to images (8).jpg
■ Added: images (8).jpg

■ Upload database image 2:

Saving images (9).jpg to images (9).jpg
■ Added: images (9).jpg

■ Upload database image 3:

Saving download (1).jpg to download (1) (3).jpg
■ Added: download (1) (3).jpg

■ Upload database image 4:

Saving images (6).jpg to images (6) (5).jpg
■ Added: images (6) (5).jpg

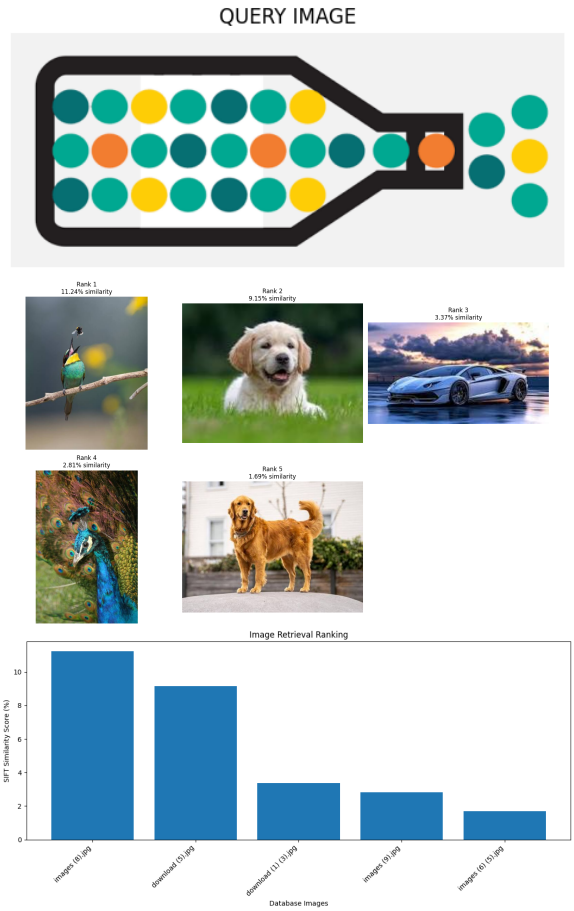
■ Upload database image 5:

Saving download.jpg to download (5).jpg
■ Added: download (5).jpg

```
=======================================
QUERY IMAGE INFORMATION
=======================================
Query image: images.png
Query keypoints: 178

=======================================
SEARCHING DATABASE
=======================================
images (8).jpg: 11.24% similarity | 20 good matches
images (9).jpg: 2.81% similarity | 5 good matches
download (1) (3).jpg: 3.37% similarity | 6 good matches
images (6) (5).jpg: 1.69% similarity | 3 good matches
download (5).jpg: 9.15% similarity | 14 good matches


=============================================
SIFT IMAGE RETRIEVAL RESULTS
=============================================
Rank 1: images (8).jpg | Similarity: 11.24% | Good Matches: 20
Rank 2: download (5).jpg | Similarity: 9.15% | Good Matches: 14
Rank 3: download (1) (3).jpg | Similarity: 3.37% | Good Matches: 6
Rank 4: images (9).jpg | Similarity: 2.81% | Good Matches: 5
Rank 5: images (6) (5).jpg | Similarity: 1.69% | Good Matches: 3
=============================================
```




```
=======================================
BEST MATCH
=======================================
Image: images (8).jpg
Similarity: 11.24%
Good SIFT matches: 20
=======================================

■ IMAGE RETRIEVAL COMPLETED!
```

NAME:MASNHI VERMA

ROLL NO:30

10. Buid a benchmarking tool comparing SIFT versus HOG descriptors on a basic object-recoginition task.

```
!pip -q install opencv-python scikit-image matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

from google.colab import files
from skimage.feature import hog
from skimage.color import rgb2gray
from PIL import Image

images = []
labels = []
filenames = []

print("============================================================")
print("SIFT vs HOG BENCHMARK")
print("============================================================")
print("Upload at least 4 images.")
print("Use at least 2 different object classes.")
print()
print("Example:")
print("car1.jpg    -> car")
print("car2.jpg    -> car")
print("bottle1.jpg -> bottle")
print("bottle2.jpg -> bottle")
print("============================================================")

for i in range(4):

    print(f"\n■ Upload IMAGE {i+1}:")

    uploaded = files.upload()
```

```
    if not uploaded:
        print("No image uploaded!")
        continue

    filename = list(uploaded.keys())[0]

    image = cv2.imread(filename)

    if image is None:
        print("Could not read image.")
        continue

    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    label = input(
        f"Enter object class for '{filename}' "
        "(example: car, bottle, person): "
    )

    images.append(image_rgb)
    labels.append(label)
    filenames.append(filename)

    print(f"■ Added: {filename} → Class: {label}")

if len(images) < 4:
    raise ValueError("Please upload at least 4 valid images.")

if len(set(labels)) < 2:
    raise ValueError(
        "Please use at least TWO different object classes."
    )

print("\n============================================================")
print("DATASET SUMMARY")
print("============================================================")

for i in range(len(images)):
    print(
        f"{i+1}. {filenames[i]} "
        f"→ {labels[i]} "
        f"→ Size: {images[i].shape[1]} x {images[i].shape[0]}"
    )

def extract_sift(image):

    start_time = time.perf_counter()

    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

    sift = cv2.SIFT_create()

    keypoints, descriptors = sift.detectAndCompute(
        gray, None
    )

    elapsed_time = time.perf_counter() - start_time

    if descriptors is None:
        descriptors = np.zeros((1, 128), dtype=np.float32)

    descriptor = np.mean(descriptors, axis=0)

    return descriptor, len(keypoints), elapsed_time

def extract_hog(image):

    start_time = time.perf_counter()

    gray = rgb2gray(image)

    gray = cv2.resize(gray, (128, 128))

    hog_descriptor = hog(
        gray,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        block_norm='L2-Hys'
    )

    elapsed_time = time.perf_counter() - start_time

    return hog_descriptor, elapsed_time

print("\n============================================================")
print("EXTRACTING SIFT FEATURES...")
print("============================================================")

sift_features = []
sift_keypoints = []
sift_times = []

for i, image in enumerate(images):

    descriptor, keypoint_count, elapsed = extract_sift(image)

    sift_features.append(descriptor)
    sift_keypoints.append(keypoint_count)
    sift_times.append(elapsed)

    print(
        f"{filenames[i]} → "
        f"{keypoint_count} keypoints, "
        f"{elapsed:.5f} seconds"
    )

print("\n============================================================")
print("EXTRACTING HOG FEATURES...")
print("============================================================")

hog_features = []
hog_times = []

for i, image in enumerate(images):

    descriptor, elapsed = extract_hog(image)

    hog_features.append(descriptor)
    hog_times.append(elapsed)

    print(
        f"{filenames[i]} → "
        f"{len(descriptor)} HOG features, "
        f"{elapsed:.5f} seconds"
    )

def euclidean_distance(a, b):
    return np.linalg.norm(a - b)

def classify_nearest(features, labels):

    predictions = []

    for i in range(len(features)):

        best_distance = float("inf")
        best_label = None

        for j in range(len(features)):

            if i == j:
                continue

            distance = euclidean_distance(
                features[i],
                features[j]
            )

            if distance < best_distance:
                best_distance = distance
                best_label = labels[j]

        predictions.append(best_label)

    return predictions

sift_predictions = classify_nearest(
    sift_features,
    labels
)

sift_correct = 0

for i in range(len(labels)):

    if sift_predictions[i] == labels[i]:
        sift_correct += 1

sift_accuracy = (
    sift_correct / len(labels)
) * 100

hog_predictions = classify_nearest(
    hog_features,
    labels
)

hog_correct = 0

for i in range(len(labels)):

    if hog_predictions[i] == labels[i]:
        hog_correct += 1

hog_accuracy = (
    hog_correct / len(labels)
) * 100

avg_sift_time = np.mean(sift_times)
avg_hog_time = np.mean(hog_times)

print("\n============================================================")
print("RECOGNITION RESULTS")
print("============================================================")

print("\nSIFT Predictions:")

for i in range(len(labels)):

    print(
        f"{filenames[i]} | "
        f"Actual: {labels[i]} | "
        f"Predicted: {sift_predictions[i]}"
    )

print("\nHOG Predictions:")

for i in range(len(labels)):

    print(
        f"{filenames[i]} | "
        f"Actual: {labels[i]} | "
        f"Predicted: {hog_predictions[i]}"
    )

print("\n============================================================")
print("FINAL BENCHMARK")
print("============================================================")

print(
    f"{'Method':<10}"
    f"{'Accuracy':<15}"
    f"{'Avg Time (sec)':<20}"
)

print("-" * 45)

print(
    f"{'SIFT':<10}"
    f"{sift_accuracy:.2f}%"
    f"{'':<8}"
    f"{avg_sift_time:.5f}"
)

print(
    f"{'HOG':<10}"
    f"{hog_accuracy:.2f}%"
    f"{'':<8}"
    f"{avg_hog_time:.5f}"
)

methods = ["SIFT", "HOG"]
accuracies = [sift_accuracy, hog_accuracy]

plt.figure(figsize=(7, 5))

plt.bar(methods, accuracies)

plt.ylabel("Accuracy (%)")
plt.xlabel("Descriptor")
plt.title("SIFT vs HOG - Recognition Accuracy")

plt.ylim(0, 100)

for i, value in enumerate(accuracies):

    plt.text(
        i,
        value + 2,
        f"{value:.2f}%",
        ha="center"
    )

plt.show()

times = [avg_sift_time, avg_hog_time]

plt.figure(figsize=(7, 5))

plt.bar(methods, times)

plt.ylabel("Average Extraction Time (seconds)")
plt.xlabel("Descriptor")
plt.title("SIFT vs HOG - Feature Extraction Speed")

for i, value in enumerate(times):

    plt.text(
        i,
        value,
        f"{value:.5f}",
        ha="center",
        va="bottom"
    )

plt.show()

plt.figure(figsize=(12, 8))

for i in range(len(images)):

    plt.subplot(2, 2, i + 1)

    plt.imshow(images[i])

    plt.title(
        f"{filenames[i]}\nClass: {labels[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()

print("\n============================================================")
print("CONCLUSION")
print("============================================================")

print(
    f"SIFT Recognition Accuracy : {sift_accuracy:.2f}%"
)

print(
    f"HOG Recognition Accuracy  : {hog_accuracy:.2f}%"
)

print(
    f"SIFT Average Time         : {avg_sift_time:.5f} sec"
)

print(
    f"HOG Average Time          : {avg_hog_time:.5f} sec"
)

print("\nBenchmark completed successfully.")
```

```
============================================================
SIFT vs HOG BENCHMARK
============================================================
Upload at least 4 images.
Use at least 2 different object classes.

Example:
car1.jpg -> car
car2.jpg -> car
bottle1.jpg -> bottle
bottle2.jpg -> bottle
============================================================

■ Upload IMAGE 1:

Saving download (1).jpg to download (1) (4).jpg
Enter object class for 'download (1) (4).jpg' (example: car, bottle, person): car
■ Added: download (1) (4).jpg → Class: car

■ Upload IMAGE 2:

Saving images (10).jpg to images (10).jpg
Enter object class for 'images (10).jpg' (example: car, bottle, person): car
■ Added: images (10).jpg → Class: car

■ Upload IMAGE 3:

Saving images (11).jpg to images (11).jpg
Enter object class for 'images (11).jpg' (example: car, bottle, person): bottle
■ Added: images (11).jpg → Class: bottle

■ Upload IMAGE 4:

Saving images (12).jpg to images (12).jpg
Enter object class for 'images (12).jpg' (example: car, bottle, person): bottle
■ Added: images (12).jpg → Class: bottle

============================================================
DATASET SUMMARY
============================================================
1. download (1) (4).jpg → car → Size: 246 x 138
2. images (10).jpg → car → Size: 399 x 501
3. images (11).jpg → bottle → Size: 360 x 360
4. images (12).jpg → bottle → Size: 447 x 447

============================================================
EXTRACTING SIFT FEATURES...
============================================================
```

download (1) (4).jpg → 315 keypoints, 0.02438 seconds
images (10).jpg → 298 keypoints, 0.07062 seconds
images (11).jpg → 50 keypoints, 0.02169 seconds
images (12).jpg → 94 keypoints, 0.04081 seconds

==
EXTRACTING HOG FEATURES...
==
download (1) (4).jpg → 8100 HOG features, 0.01231 seconds
images (10).jpg → 8100 HOG features, 0.00998 seconds
images (11).jpg → 8100 HOG features, 0.01281 seconds
images (12).jpg → 8100 HOG features, 0.01861 seconds

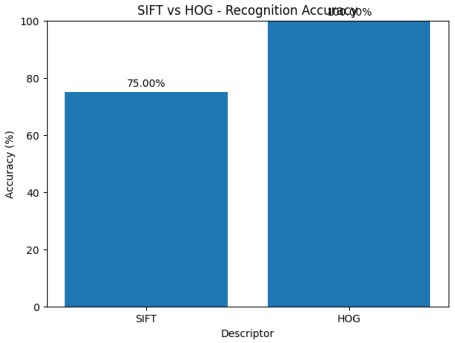
==
RECOGNITION RESULTS
==

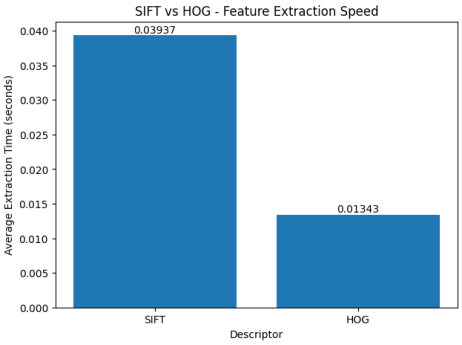
SIFT Predictions:
download (1) (4).jpg | Actual: car | Predicted: car
images (10).jpg | Actual: car | Predicted: car
images (11).jpg | Actual: bottle | Predicted: bottle
images (12).jpg | Actual: bottle | Predicted: car

HOG Predictions:
download (1) (4).jpg | Actual: car | Predicted: car
images (10).jpg | Actual: car | Predicted: car
images (11).jpg | Actual: bottle | Predicted: bottle
images (12).jpg | Actual: bottle | Predicted: bottle

==
FINAL BENCHMARK
==
Method Accuracy Avg Time (sec)

SIFT 75.00% 0.03937
HOG 100.00% 0.01343













==
CONCLUSION
==
SIFT Recognition Accuracy : 75.00%
HOG Recognition Accuracy : 100.00%
SIFT Average Time : 0.03937 sec
HOG Average Time : 0.01343 sec

Benchmark completed successfully.